PHASE 3: SUBMISSION CHECKLIST/SIGN OFF SHEET

GROUP#: 1 GROUP NAME: DKAA Crew

Deliverables:

- Revised and Consistent Documentation
- Implementation of classes and queries in Python (one per student) using ORM

Group Assessment

- · Group Status Report

We have each reviewed the contents of this deliverable.

Phase Leader Dominic Mattiello _____

Phase Recorder Aakash Dawani _____

Phase Checker Arslan Bisharat _____

Technical Advisor Kanishq Patel _____

1. Introduction

This document details the Music Streaming Service Database design. The platform connects users to music, podcasts, and playlists. It offers personalized experiences tailored to individual user preferences. The database manages users, content, and user interactions efficiently.

- Organization: The service provides digital music and audio streaming. It connects listeners, artists, and content creators seamlessly. Millions of tracks are available on-demand for users. The platform enhances personalized and engaging music experiences. It supports multiple music genres and global artists. The primary goal is to connect artists with audiences.
- Expert User: Expert users are data analysts, engineers, and managers. Analysts use data to optimize user experience and engagement. Product managers enhance platform features and user interfaces. Engineers manage data flow, quality, and system performance. They ensure music recommendations are accurate and relevant. Expert users track content performance and user behavior.
- Group Contact: Our database management, analytics, and support teams collaborate closely to ensure the smooth operation of the platform. They work together to maintain data accuracy, resolve system issues promptly, and optimize recommendations to enhance user satisfaction. The support team is dedicated to handling user queries and feedback efficiently, facilitating seamless communication between users and technical teams. By working together, these teams play a crucial role in improving artist-platform interactions.

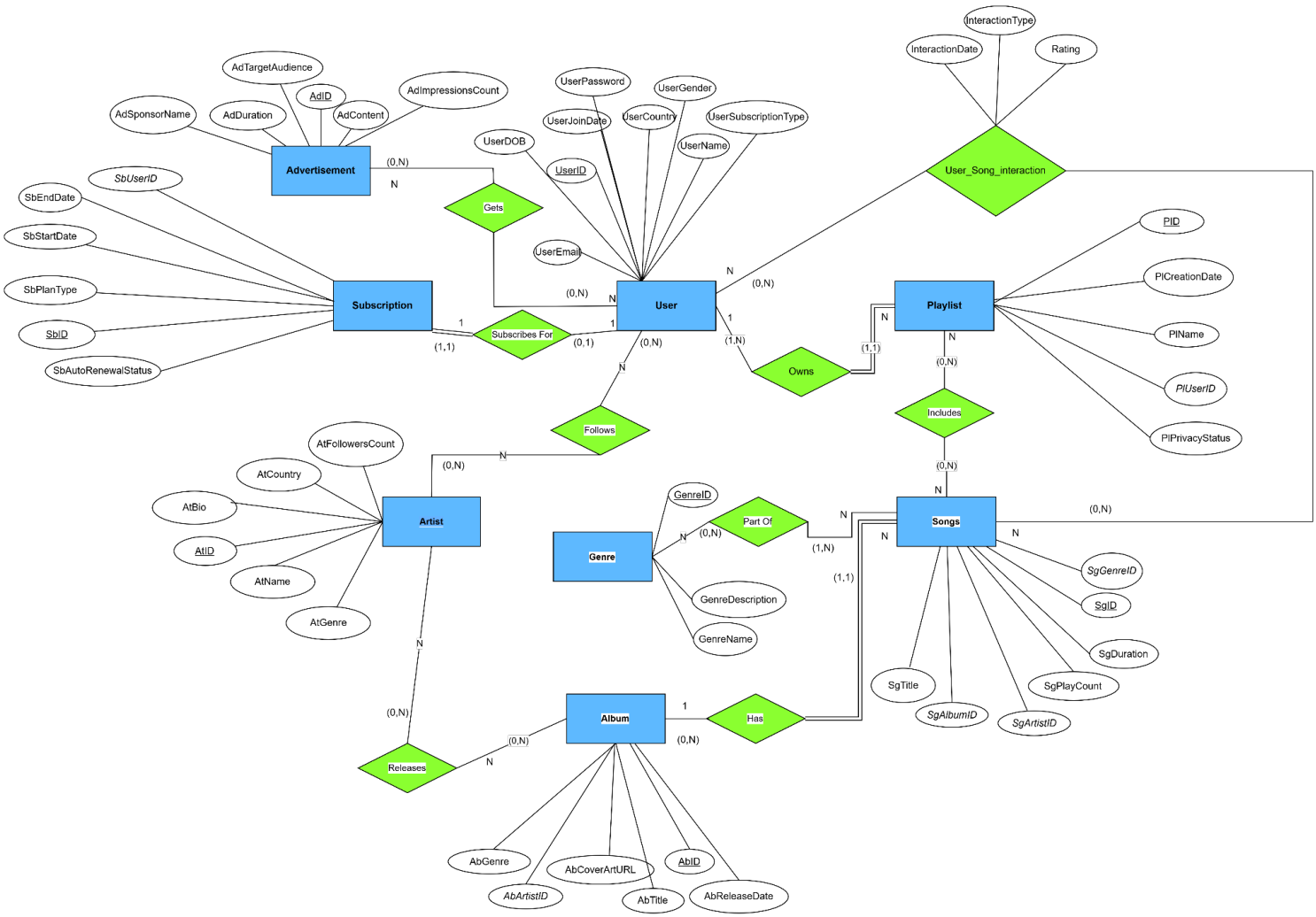
2. Description:

The Music Streaming Service Database stores key information about users, artists, songs, playlists, and other related entities to make music discovery and personalized user experiences possible. It contains data on users, artists, albums, songs, playlists, genres, advertisements, and how they interact with each other.

- Users are individuals who listen to albums, create playlists, and interact with songs. Each user has a unique ID, username, email, password, subscription type, country, date of birth, gender, and join date. Users can create multiple playlists but each playlist is created by only one user. Users can also interact with multiple songs.
- Playlists are collections of songs curated by users. Each playlist has a unique playlist ID, a name, the creation date, and a privacy status (public, private, or unlisted). A playlist can contain many songs.
- Songs are individual music tracks that users can play, like, or add to playlists. Each song has a unique song ID, title, duration, and play count. Songs are linked to one artist and one album. Songs can also belong to multiple genres, meaning a song can be categorized under several music styles, and each genre can include multiple songs.
- Artists are the creators of music on the platform. Each artist has a unique artist ID, name, genre, biography, country of origin, and number of followers. An artist can release multiple albums but each album must have at least one artist. Many Users can follow an Artist.

- Albums are collections of songs released by artists. Each album has a unique album ID, title, release date, genre, and cover art. An album can contain multiple songs but each song belongs to only one album.
- Songs can be categorized into different Genres, such as pop, rock, or jazz. Each genre has a unique genre ID, a name, and a description. A song can belong to multiple genres, and each genre can include many songs.
- Users who want to use this streaming platform open an account and select a subscription. Subscriptions represent the active plans users have on the platform. Each subscription has a unique ID, plan type (Basic, Standard, or Premium), start and end dates, and an auto-renewal status. The Basic plan is free, the Standard plan costs 5 USD per month, and the Premium plan is 10 USD per month. Note that a user can have only one active paid subscription at a time, or none.
- Lastly the app also keeps the count on the advertisements that are promotional content shown to unpaid users. Each ad has a unique Ad ID, content details, target audience, duration, sponsor name, and number of times it's been shown to users (impressions). Ads are designed to appear while users listen to music. A user gets to see multiple ad and ad can be shown to multiple users.

3. ER Diagram



4. ER Diagram Uncaptured Constraints

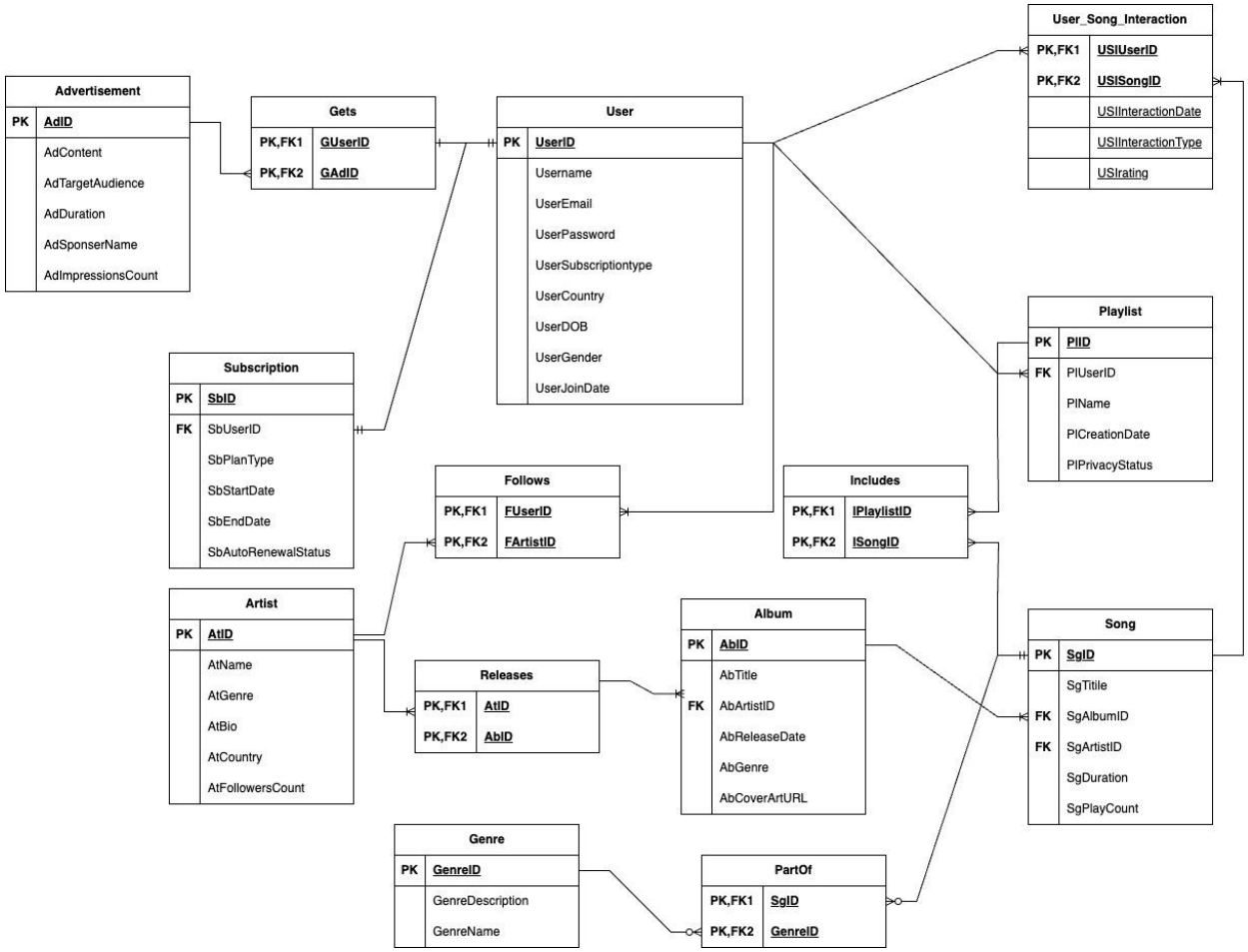
- SubscriptionType can have one of the following values: Basic, Standard, or Premium where Basic means free, Standard costs 5 USD and Premium is for 10

USD. A user can have only one active paid subscription at a time, or no subscription (free plan).

- PrivacyStatus for playlists can have one of the following values: Public, Private, or Unlisted.
- Rating must be on a scale of 1 to 5, where 1 is the lowest and 5 is the highest.
- PlayCount cannot be negative and must increment only through valid user interactions.
- AutoRenewalStatus for subscriptions can be either Yes or No.
- Duration of songs must be specified in seconds and cannot be zero or negative.
- Country should be a valid ISO country code, e.g., US, UK, CA.
- Date fields such as JoinDate, ReleaseDate, and CreationDate must follow the YYYY/DD/MM format.
- Email addresses for users must be unique and in a valid email format.
- PlanType for subscriptions can be Basic, Standard, or Premium only.
- ImpressionsCount for advertisements must be a non-negative integer.
- Account balances and financial fields (if any) should be in USD.
- Song Titles cannot be blank and must have a minimum length of 1 character.
- InteractionType can only be one of the following: Play, Like, Dislike, Rate.
- Gender for users can be Male, Female, Non-Binary, or Prefer not to say.

The figure below shows the ER diagram of the Music Streaming Service Database.

5. Relational Schema



5.1 Relational Schema with Referential Integrity

- User (<u>UserID</u>, UserName, UserEmail, UserPassword, UserSubscriptiontype, UserCountry, UserDOB, UserGender, UserJoinDate)
- Artist (<u>AtID</u>, AtName, AtGenre, AtBio, AtCountry, AtFollowersCount)
- Album (<u>AbID</u>, AbTitle, AbReleaseDate, AbGenre, AbCoverArtURL)

- Song (SgID, SgTitle, SgAlbumID, SgDuration, SgPlayCount)

  foreign key (SgAlbumID) references Album(AbID)

- Playlist (PlID, PlUserID, PlName, PlCreationDate, PlPrivacyStatus)

  foreign key (PlUserID) references User(UserID)

- Subscription (SbID, SbUserID, SbPlanType, SbStartDate, SbEndDate, SbAutoRenewalStatus)

  foreign key (SbUserID) references User (UserID)

- Genre (GenreID, GenreName, GenreDescription)
- Advertisement (AdID, AdContent, AdTargetAudience, AdDuration, AdSponsorName, AdImpressionsCount)
- Includes (IPlaylistID, ISongID).

  foreign key (ISongID) references Song (SgID)

  foreign key (IPlaylistID) references Playlist (PlID)

- Follows (FUserID, FArtistID).

  foreign key (FUserID) references User (UserID)

  foreign key (FArtistID) references Artist (AtID)

- Gets (GAdID, GUserID).

  foreign key (GUserID) references User (UserID)

foreign key (GAdID) references Advertisement (AdID)

- User_Song_Interaction (<u>UserID, SongID,</u> InteractionType, InteractionDate, Rating)

  foreign key (UserID) references User (UserID)

  foreign key (SongID) references Song (SgID)

- Releases (<u>AtID, AbID</u>)

  Foreign key(AtID) references Artist(AtID)

  Foreign key(AbID) references Album(AbID)

- PartOf (<u>GenreID, SgID</u>)

  Foreign key(GenreID) references Genre(GenreID)

  Foreign key(SgID) references Songs(SgID)

  Key Annotations:

  - <u>Underline</u> for Primary Key (PK)

5.2 Relational Table Details

The relational schema given in Section 5.1 was mapped into the following tables in the Music Streaming Service Database. Primary keys have been underlined. Tables that have multiple attributes underlined represent composite keys.

Table Name	Attribute	Description
User	<u>UserID</u>	Unique user ID
	UserName	Username of the user
	UserEmail	Email address of the user must be unique
	UserPassword	Password for a user account
	UserSubscriptionType	Type of subscription: Basic, Standard, or Premium
	UserCountry	ISO country code of the user
	UserDOB	Date of birth of the user (YYYY/DD/MM)
	UserGender	Gender of the user: Male, Female, Non-Binary, or Prefer not to say

	UserJoinDate	Date the user joined the platform (YYYY/DD/MM)
Artist	AtID	Unique artist ID
	AtName	Name of the artist
	AtGenre	Main genre of the artist
	AtBio	Short biography of the artist
	AtCountry	Country of origin of the artist
	AtFollowersCount	Number of followers the artist has on the platform
Album	AbID	Unique album ID
	AbTitle	Title of the album

	AbArtistID	Artist of the album (Foreign key to Artist)
	AbReleaseDate	Release date of the album (YYYY/DD/MM)
	AbGenre	Genre of the album
	AbCoverArtURL	URL of the album cover art
Song	SgID	Unique song ID
	SgTitle	Title of the song
	SgAlbumID	Unique album ID the song belongs to (Foreign key to Album)
	SgArtistID	Unique artist ID who performed the song (Foreign Key to Artist)
	SgDuration	Duration of the song in seconds

	SgPlayCount	Number of times the song has been played
Playlist	PlID	Unique playlist ID
	PlUserID	Unique user ID who created the playlist (Foreign key to User)
	PlName	Name of the playlist
	PlCreationDate	Date the playlist was created (YYYY/DD/MM)
	PlPrivacyStatus	Privacy status of the playlist: Public, Private, or Unlisted
Subscription	SbID	Unique subscription ID
	SbUserID	Unique user ID with the subscription (Foreign key from User)

	SbPlanType	Plan type of the subscription: Basic, Standard, or Premium
	SbStartDate	Start date of the subscription (YYYY/DD/MM)
	SbEndDate	End date of the subscription (YYYY/DD/MM)
	SbAutoRenewalStatus	Whether the subscription renews automatically: Yes (1) or No (0)
Genre	GenreID	Unique genre ID
	GenreName	Name of the genre
	GenreDescription	Description of the genre
Advertisement	AdID	Unique advertisement ID

	AdContent	Content details of the advertisement
	AdTargetAudience	Target audience for the advertisement
	AdDuration	Duration of the advertisement in seconds
	AdSponsorName	Name of the sponsor of the advertisement
	AdImpressionsCount	Number of times the advertisement has been shown to users
Includes	IPlaylistID	Unique playlist ID (Foreign key to Playlist)
	ISongID	Unique song ID (Foreign key to Song)
Gets	GAdID	Unique advertisement ID (Foreign key to Advertisement)
	GUserID	Unique user ID (Foreign key to User)

Follows	FUserID	Unique user ID (Foreign key to User)
	FArtistID	Unique artist ID (Foreign key to Artist)
SongGenre	SgID	Unique song ID (Foreign key to Song)
	GenreID	Unique genre ID (Foreign key to Genre)
User_Song_Interaction	UserID	Unique user ID (Foreign key to User)
	SongID	Unique song ID (Foreign key to Song)
	InteractionType	Type of interaction: Play, Like, Dislike, or Rate
	InteractionDate	Date of the interaction (YYYY/DD/MM)
	Rating	User's rating of the song (1 to 5, if applicable)

Releases	AtID	Unique Artist ID (Foreign key to Artist)
	AbID	Unique Album ID (Foreign key to Album)
PartOf	GenreID	Unique Genre ID (Foreign key to Genre)
	SgID	Unique Song ID (Foreign key to Song)

6. Queries

The following table summarizes the queries in the Music Database.

Query Name	Description	Output	Relations Accessed
numSongsInteracted	For each user, retrieve the total number of distinct songs they interacted with (played, liked, or rated). Sort the results by the total interactions in descending order.	● Username ● NumSongsInteracted	● Users ● UserSongInteractions
avgSongRatingByArtist	Calculate the average rating for songs by each artist. Include only songs that have been rated. Sort the results by the average rating in descending order.	● ArtistName ● AverageRating	● Artists ● Songs ● UserSongInteractions
mostPopularGenre	Retrieve the most popular genre based on the total number of song plays. Sort the results by total play count in descending order and display only the top genre.	● GenreName ● TotalPlayCount	● Genres ● Songs ● Albums
playlistCountByPrivacyStatus	Shows the songs played by users who have subscribed to the premium plan. Grouped by Song title. Sort by the result by the count of the play in descending order	● Song Title ● Play Count	● Songs ● Users ● Subscriptions ● usersonginteractions

Table Creation

```sql
-- Users Table
CREATE TABLE Users (
    UserID SERIAL PRIMARY KEY,
    Username VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    Password VARCHAR(255) NOT NULL,
    SubscriptionType VARCHAR(10),
    Country CHAR(2),
    DateOfBirth DATE,
    Gender VARCHAR(20),
    JoinDate DATE DEFAULT CURRENT_DATE
);

-- Artists Table
CREATE TABLE Artists (
    ArtistID SERIAL PRIMARY KEY,
    ArtistName VARCHAR(100) NOT NULL,
    Genre VARCHAR(30),
    Bio TEXT,
    Country CHAR(2),
    FollowersCount INT DEFAULT 0
);

-- Albums Table
CREATE TABLE Albums (
    AlbumID SERIAL PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    ArtistID INT,
    ReleaseDate DATE,
    Genre VARCHAR(30),
    CoverArtURL VARCHAR(255),
    FOREIGN KEY (ArtistID) REFERENCES Artists(ArtistID)
);

-- Songs Table
CREATE TABLE Songs (
```

```sql
    SongID SERIAL PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    AlbumID INT,
    ArtistID INT,
    Duration INT,
     PlayCount INT DEFAULT 0,
    FOREIGN KEY (AlbumID) REFERENCES Albums(AlbumID),
    FOREIGN KEY (ArtistID) REFERENCES Artists(ArtistID)
);

-- Playlists Table
CREATE TABLE Playlists (
    PlaylistID SERIAL PRIMARY KEY,
    UserID INT,
    Name VARCHAR(100) NOT NULL,
    CreationDate DATE DEFAULT CURRENT_DATE,
    PrivacyStatus VARCHAR(10) DEFAULT 'Private',
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);

-- Subscriptions Table
CREATE TABLE Subscriptions (
    SubscriptionID SERIAL PRIMARY KEY,
    UserID INT,
    PlanType VARCHAR(10) NOT NULL,
    StartDate DATE NOT NULL,
    EndDate DATE,
    AutoRenewal BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);

-- Genres Table
CREATE TABLE Genres (
    GenreID SERIAL PRIMARY KEY,
    Name VARCHAR(50) UNIQUE NOT NULL,
    Description TEXT
);

-- Advertisements Table
```

```sql
CREATE TABLE Advertisements (
    AdID SERIAL PRIMARY KEY,
    Content TEXT NOT NULL,
    TargetAudience VARCHAR(50),
    Duration INT,
     SponsorName VARCHAR(100),
    ImpressionsCount INT DEFAULT 0
);

-- PlaylistSongs Table (formerly Includes)
CREATE TABLE PlaylistSongs (
    PlaylistID INT,
    SongID INT,
    AddedDate DATE DEFAULT CURRENT_DATE,
    PRIMARY KEY (PlaylistID, SongID),
    FOREIGN KEY (PlaylistID) REFERENCES Playlists(PlaylistID),
    FOREIGN KEY (SongID) REFERENCES Songs(SongID)
);

-- UserArtistFollows Table (formerly Follows)
CREATE TABLE UserArtistFollows (
    UserID INT,
    ArtistID INT,
    FollowDate DATE DEFAULT CURRENT_DATE,
    PRIMARY KEY (UserID, ArtistID),
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (ArtistID) REFERENCES Artists(ArtistID)
);

-- UserAdImpressions Table (formerly Gets)
CREATE TABLE UserAdImpressions (
    AdID INT,
    UserID INT,
    ImpressionDate DATE DEFAULT CURRENT_DATE,
    PRIMARY KEY (AdID, UserID),
    FOREIGN KEY (AdID) REFERENCES Advertisements(AdID),
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);
```

```sql
-- UserSongInteractions Table
CREATE TABLE UserSongInteractions (
    UserID INT,
    SongID INT,
    InteractionType VARCHAR(10) NOT NULL,
    InteractionDate DATE DEFAULT CURRENT_DATE,
    Rating INT CHECK (Rating BETWEEN 1 AND 5),
    PRIMARY KEY (UserID, SongID, InteractionType),
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (SongID) REFERENCES Songs(SongID)
);
```

Insert

```sql
INSERT INTO Users (UserID, Username, Email, Password, SubscriptionType, Country, DateOfBirth, Gender, JoinDate) VALUES
(1, 'musiclover123', 'musiclover123@email.com', 'password123', 'Premium', 'US', '1995-01-20', 'Female', '2022-05-10'),
(2, 'rockfan456', 'rockfan456@email.com', 'securepassword', 'Basic', 'CA', '1988-06-15', 'Male', '2021-08-22'),
(3, 'popenthusiast789', 'pop789@email.com', 'musicfanatic', 'Standard', 'UK', '1990-09-08', 'Non-Binary', '2023-01-18'),
(4, 'jazzlover001', 'jazz001@email.com', 'jazzytunes', 'Premium', 'FR', '1975-12-02', 'Male', '2020-11-05'),
(5, 'countryfan234', 'country234@email.com', 'countryroads', 'Basic', 'US', '1992-03-28', 'Female', '2022-09-12'),
(6, 'hiphophead567', 'hiphop567@email.com', 'hiphopbeats', 'Standard', 'DE', '1985-07-19', 'Male', '2021-04-03'),
(7, 'classicalnut890', 'classical890@email.com', 'classicalmusic', 'Premium', 'IT', '1998-10-14', 'Female', '2023-02-25'),
(8, 'indielover012', 'indie012@email.com', 'indierocks', 'Basic', 'AU', '1980-05-09', 'Non-Binary', '2020-06-17'),
(9, 'electronicfan345', 'electronic345@email.com', 'edmlover', 'Standard', 'JP', '1994-08-01', 'Male', '2022-12-09'),
```

```
(10, 'worldmusicguru678', 'worldmusic678@email.com', 'globaltunes', 'Premium', 'BR',
'1972-11-27', 'Female', '2021-07-21'),
(11, 'metalhead901', 'metalhead901@email.com', 'heavymetalfan', 'Basic', 'US',
'1987-02-13', 'Prefer not to say', '2020-03-04'),
(12, 'folkfan023', 'folkfan023@email.com', 'folksongs', 'Standard', 'CA', '1999-04-
29', 'Female', '2023-10-16'),
(13, 'blueslover145', 'blues145@email.com', 'bluesmusic', 'Premium', 'UK', '1970-09-
06', 'Male', '2022-08-28'),
(14, 'punkrocker256', 'punkrocker256@email.com', 'punkrockfan', 'Basic', 'DE',
'1983-01-21', 'Non-Binary', '2021-05-19'),
(15, 'reggaefan367', 'reggae367@email.com', 'reggaemusic', 'Standard', 'JM', '1996-
06-12', 'Female', '2023-09-03'),
(16, 'mlawillef', 'khardwinf@quantcast.com', 'fO6@#r2>=D&0', 'Premium', 'VU', '1970-
12-09', 'Non-binary', '2018-09-22'),
(17, 'tgioanig', 'cwoodisong@businesswire.com', 'gW3)_8N`', 'Premium', 'UZ', '1981-
06-11', 'Female', '2012-01-28'),
(18, 'ttulletth', 'dturgooseh@fotki.com', 'dL6TF8NAM', 'Premium', 'RS', '1950-01-
20', 'Female', '2020-07-19'),
(19, 'jkitchinghani', 'cmckeadyi@omniture.com', '"yU1}Q#/J+,"', 'Premium', 'HR',
'1993-03-01', 'Female', '2018-01-19'),
(20, 'cthistletonj', 'myellowleaj@yellowbook.com', 'kR1Co4j{?sN', 'Standard', 'ID',
'1969-04-26', 'Female', '2015-09-08'),
(21, 'woheffernank', 'tciraldok@nydailynews.com', 'tM1vcuaL@E=', 'Basic', 'CN',
'1973-08-25', 'Male', '2016-07-29'),
(22, 'glegganl', 'adeall@taobao.com', 'xC2n>1%dqaTQ1rC', 'Standard', 'CN', '1988-03-
15', 'Male', '2015-10-23'),
(23, 'jjurewiczm', 'mwottonm@dagondesign.com', 'xZ3P_O7e(!7%6s*', 'Standard', 'RS',
'1968-06-16', 'Female', '2014-04-08'),
(24, 'ddurgann', 'hextencen@mysql.com', 'qA80xHRjAa$', 'Basic', 'RU', '1985-06-05',
'Male', '2014-10-12'),
(25, 'clittello', 'gworsnipo@ehow.com', 'lK8RKq\4d', 'Standard', 'PT', '1975-07-04',
'Male', '2015-11-22'),
(26, 'lmactrustramp', 'bhayseyp@japanpost.jp', 'uN2\!FcEqg', 'Basic', 'AR', '1990-
11-11', 'Male', '2014-03-12'),
(27, 'kjeenesq', 'hsamwayesq@youtu.be', 'uT7ZP9ScKQ+Y8fn', 'Standard', 'FR', '1952-
04-05', 'Male', '2015-02-15'),
(28, 'mwillgrassr', 'thandrokr@gov.uk', '"eK3la""(I|', 'Premium', 'CN', '1985-09-
14', 'Male', '2011-05-17'),
(29, 'hlumberss', 'ccamplings@newyorker.com', 'oS6g=Nd56P', 'Premium', 'PL', '2000-
```

```
04-30', 'Male', '2019-10-18'),
(30, 'bgyurkot', 'rcandwellt@trellian.com', 'eI3@SW6B|_<', 'Premium', 'SE', '1951-
08-28', 'Polygender', '2017-12-12'),
(31, 'vharfordu', 'mpieteschu@mozilla.com', 'hJ4/wB.$sUJeg(z', 'Premium', 'CN',
'1986-08-07', 'Female', '2012-06-27'),
(32, 'mduffv', 'itacev@indiatimes.com', 'qF7eiApDVq', 'Basic', 'KZ', '1975-07-21',
'Male', '2014-05-02'),
(33, 'fpizziew', 'jpendrighw@usa.gov', 'bL0*D9.>D)|XOyAe', 'Premium', 'AM', '1977-
02-03', 'Female', '2019-08-23'),
(34, 'wdutteridgex', 'cpartonx@youku.com', '"rU2C6A\vvp*x""pP', 'Standard', 'NL',
'1970-03-27', 'Female', '2014-04-02'),
(35, 'oreilingery', 'vwillavizey@example.com', 'uM8B$uVwQyT', 'Premium', 'BG',
'1952-12-08', 'Female', '2011-05-11');


INSERT INTO Artists (ArtistID, ArtistName, Genre, Bio, Country, FollowersCount)
VALUES
(1, 'The Beatles', 'Rock', 'The Beatles were an English rock band, formed in
Liverpool in 1960.', 'UK', 1000000),
(2, 'Beyoncé', 'Pop', 'Beyoncé Giselle Knowles-Carter is an American singer,
songwriter, producer, and actress.', 'US', 500000),
(3, 'Taylor Swift', 'Pop', 'Taylor Alison Swift is an American singer-songwriter.',
'US', 750000),
(4, 'Ed Sheeran', 'Pop', 'Edward Christopher Sheeran is an English singer-
songwriter, record producer, musician, and actor.', 'UK', 600000),
(5, 'Drake', 'Hip-hop', 'Aubrey Drake Graham is a Canadian rapper, singer,
songwriter, record producer, actor, and entrepreneur.', 'CA', 900000),
(6, 'Adele', 'Pop', 'Adele Laurie Blue Adkins MBE is an English singer-songwriter.',
'UK', 800000),
(7, 'Rihanna', 'Pop', 'Robyn Rihanna Fenty NH is a Barbadian singer, actress, and
businesswoman.', 'BB', 400000),
(8, 'BTS', 'K-pop', 'BTS, also known as the Bangtan Boys, is a seven-member South
Korean boy band.', 'KR', 1200000),
(9, 'Justin Bieber', 'Pop', 'Justin Drew Bieber is a Canadian singer.', 'CA',
300000),
(10, 'Billie Eilish', 'Pop', 'Billie Eilish Pirate Baird O''Connell is an American
singer-songwriter.', 'US', 550000),
(11, 'Fianna McKerron', 'Rock', 'Cras non velit nec nisi vulputate nonummy. Maecenas
tincidunt lacus at velit. Vivamus vel nulla eget eros elementum pellentesque.',
```

```
'CN', 286479),
(12, 'Arlette Guice', 'Pop', 'Phasellus in felis. Donec semper sapien a libero. Nam dui.', 'PT', 770786),
(13, 'Loraine Grigoryov', 'Electronic', 'Pellentesque at nulla. Suspendisse potenti. Cras in purus eu magna vulputate luctus.', 'CN', 392209),
(14, 'Sunny Brader', 'Rock', 'Curabitur at ipsum ac tellus semper interdum. Mauris ullamcorper purus sit amet nulla. Quisque arcu libero, rutrum ac, lobortis vel, dapibus at, diam.', 'AR', 307793),
(15, 'Larina Sacaze', 'Hip Hop', 'Vestibulum ac est lacinia nisi venenatis tristique. Fusce congue, diam id ornare imperdiet, sapien urna pretium nisl, ut volutpat sapien arcu sed augue. Aliquam erat volutpat.', 'KR', 32666),
(16, 'Orel Taffs', 'Electronic', 'Sed ante. Vivamus tortor. Duis mattis egestas metus.', 'NO', 795270),
(17, 'Sylvan Jancey', 'Pop', 'Aenean fermentum. Donec ut mauris eget massa tempor convallis. Nulla neque libero, convallis eget, eleifend luctus, ultricies eu, nibh.', 'RU', 67665),
(18, 'Veradis Ellen', 'Electronic', 'Morbi non lectus. Aliquam sit amet diam in magna bibendum imperdiet. Nullam orci pede, venenatis non, sodales sed, tincidunt eu, felis.', 'AF', 964361),
(19, 'Amalie Ouldcott', 'Electronic', 'Etiam vel augue. Vestibulum rutrum rutrum neque. Aenean auctor gravida sem.', 'TZ', 221584),
(20, 'Esma Saben', 'Rock', 'Proin eu mi. Nulla ac enim. In tempor, turpis nec euismod scelerisque, quam turpis adipiscing lorem, vitae mattis nibh ligula nec sem.', 'CZ', 889576),
(21, 'Hilde Cleef', 'Pop', 'Praesent blandit. Nam nulla. Integer pede justo, lacinia eget, tincidunt eget, tempus vel, pede.', 'ID', 833309),
(22, 'Gray Holsall', 'Electronic', 'Praesent id massa id nisl venenatis lacinia. Aenean sit amet justo. Morbi ut odio.', 'MN', 563146),
(23, 'Cristin Brocklesby', 'Jazz', 'Duis bibendum, felis sed interdum venenatis, turpis enim blandit mi, in porttitor pede justo eu massa. Donec dapibus. Duis at velit eu est congue elementum.', 'PH', 748711),
(24, 'Chevy Rickaert', 'Rock', 'Curabitur in libero ut massa volutpat convallis. Morbi odio odio, elementum eu, interdum eu, tincidunt in, leo. Maecenas pulvinar lobortis est.', 'JP', 73269),
(25, 'Corny Ross', 'Hip Hop', 'Fusce posuere felis sed lacus. Morbi sem mauris, laoreet ut, rhoncus aliquet, pulvinar sed, nisl. Nunc rhoncus dui vel sem.', 'CN', 101578),
(26, 'Arabel Poveleye', 'Pop', 'Nullam porttitor lacus at turpis. Donec posuere metus vitae ipsum. Aliquam non mauris.', 'VE', 266886),
```

```
(27, 'Herbert Southworth', 'Hip Hop', 'In hac habitasse platea dictumst. Morbi
vestibulum, velit id pretium iaculis, diam erat fermentum justo, nec condimentum
neque sapien placerat ante. Nulla justo.', 'TD', 56757),
(28, 'Shaughn Minster', 'Jazz', 'Nam ultrices, libero non mattis pulvinar, nulla
pede ullamcorper augue, a suscipit nulla elit ac nulla. Sed vel enim sit amet nunc
viverra dapibus. Nulla suscipit ligula in lacus.', 'PT', 284657),
(29, 'Cal Rothera', 'Jazz', 'Phasellus in felis. Donec semper sapien a libero. Nam
dui.', 'ID', 848688),
(30, 'Hilarius Gouda', 'Pop', 'Cras non velit nec nisi vulputate nonummy. Maecenas
tincidunt lacus at velit. Vivamus vel nulla eget eros elementum pellentesque.',
'PT', 890616),
(31, 'Hart Juorio', 'Pop', 'In sagittis dui vel nisl. Duis ac nibh. Fusce lacus
purus, aliquet at, feugiat non, pretium quis, lectus.', 'CN', 669383),
(32, 'Thorsten Hardey', 'Hip Hop', 'In congue. Etiam justo. Etiam pretium iaculis
justo.', 'ID', 353201),
(33, 'Arline Hearst', 'Pop', 'Suspendisse potenti. In eleifend quam a odio. In hac
habitasse platea dictumst.', 'CN', 902969),
(34, 'Dwain Siburn', 'Pop', 'Suspendisse potenti. In eleifend quam a odio. In hac
habitasse platea dictumst.', 'CN', 527745),
(35, 'Bone Quinton', 'Hip Hop', 'Proin leo odio, porttitor id, consequat in,
consequat ut, nulla. Sed accumsan felis. Ut at dolor quis odio consequat varius.',
'PH', 803069);


INSERT INTO albums (albumid, title, ArtistID, ReleaseDate, Genre, CoverArtURL)
VALUES
(1, 'Abbey Road', 1, '1969-09-26', 'Rock', 'https://example.com/abbey_road.jpg'),
(2, 'Lemonade', 2, '2016-04-23', 'Pop', 'https://example.com/lemonade.jpg'),
(3, '1989', 3, '2014-10-27', 'Pop', 'https://example.com/1989.jpg'),
(4, '÷ (Divide)', 4, '2017-03-03', 'Pop', 'https://example.com/divide.jpg'),
(5, 'Scorpion', 5, '2018-06-29', 'Hip Hop', 'https://example.com/scorpion.jpg'),
(6, '25', 6, '2015-11-20', 'Pop', 'https://example.com/25.jpg'),
(7, 'Anti', 7, '2016-01-28', 'Pop', 'https://example.com/anti.jpg'),
(8, 'Love Yourself: Tear', 8, '2018-05-18', 'K-pop',
'https://example.com/love_yourself_tear.jpg'),
(9, 'Purpose', 9, '2015-11-13', 'Pop', 'https://example.com/purpose.jpg'),
(10, 'When We Fall Asleep, Where Do We Go?', 10, '2019-03-29', 'Pop',
'https://example.com/when_we_fall_asleep.jpg'),
(11, 'Rock Solid', 11, '2023-03-29', 'Rock', 'https://example.com/cover11.jpg'),
```

```
(12, 'Pop Classics', 12, '2020-11-11', 'Pop', 'https://example.com/cover12.jpg'),
(13, 'Beats of Tomorrow', 13, '2021-07-20', 'Electronic',
'https://example.com/cover13.jpg'),
(14, 'Rock the World', 14, '2022-02-25', 'Rock', 'https://example.com/cover14.jpg'),
(15, 'Urban Pulse', 15, '2023-04-01', 'Hip Hop', 'https://example.com/cover15.jpg'),
(16, 'Norwegian Nights', 16, '2021-06-14', 'Electronic',
'https://example.com/cover16.jpg'),
(17, 'Pop Revolution', 17, '2020-10-10', 'Pop', 'https://example.com/cover17.jpg'),
(18, 'Afghan Soundscapes', 18, '2022-08-21', 'Electronic',
'https://example.com/cover18.jpg'),
(19, 'Tanzanian Beats', 19, '2023-05-15', 'Electronic',
'https://example.com/cover19.jpg'),
(20, 'Czech Harmony', 20, '2021-12-05', 'Rock', 'https://example.com/cover20.jpg'),
(21, 'Indonesian Melodies', 21, '2020-03-03', 'Pop',
'https://example.com/cover21.jpg'),
(22, 'Mongolian Folk', 22, '2023-06-18', 'Electronic',
'https://example.com/cover22.jpg'),
(23, 'Philippine Jazz', 23, '2021-11-22', 'Jazz',
'https://example.com/cover23.jpg'),
(24, 'Japanese Fusion', 24, '2022-04-12', 'Rock',
'https://example.com/cover24.jpg'),
(25, 'China Soundtrack', 25, '2020-09-30', 'Hip Hop',
'https://example.com/cover25.jpg'),
(26, 'Venezuelan Rhythms', 26, '2021-02-02', 'Pop',
'https://example.com/cover26.jpg'),
(27, 'Chad Beats', 27, '2023-01-15', 'Hip Hop', 'https://example.com/cover27.jpg'),
(28, 'Portuguese Jazz', 28, '2021-05-10', 'Jazz',
'https://example.com/cover28.jpg'),
(29, 'Indonesian Grooves', 29, '2022-03-29', 'Jazz',
'https://example.com/cover29.jpg'),
(30, 'Classic Hits', 30, '2021-04-14', 'Pop', 'https://example.com/cover30.jpg'),
(31, 'China Nights', 31, '2022-01-18', 'Pop', 'https://example.com/cover31.jpg'),
(32, 'Hardey Beats', 32, '2023-02-01', 'Hip Hop',
'https://example.com/cover32.jpg'),
(33, 'Hearst Harmonies', 33, '2021-08-05', 'Pop',
'https://example.com/cover33.jpg'),
(34, 'Siburn Sounds', 34, '2023-07-17', 'Pop', 'https://example.com/cover34.jpg'),
(35, 'Philippine Fusion', 35, '2022-12-30', 'Hip Hop',
'https://example.com/cover35.jpg');
```

```sql
INSERT INTO Songs (SongID, Title, AlbumID, ArtistID, Duration, PlayCount) VALUES
(1, 'Here Comes the Sun', 1, 1, 186, 1500000),
(2, 'Formation', 2, 2, 187, 1200000),
(3, 'Shake It Off', 3, 3, 219, 1800000),
(4, 'Shape of You', 4, 4, 233, 2000000),
(5, 'God''s Plan', 5, 5, 199, 1600000),
(6, 'Hello', 6, 6, 295, 1900000),
(7, 'Work', 7, 7, 216, 1400000),
(8, 'Fake Love', 8, 8, 295, 1700000),
(9, 'Sorry', 9, 9, 198, 1300000),
(10, 'Bad Guy', 10, 10, 194, 1500000),
(11, 'Come Together', 1, 1, 257, 1300000),
(12, 'Hold Up', 2, 2, 215, 1100000),
(13, 'Blank Space', 3, 3, 230, 1600000),
(14, 'Thinking Out Loud', 4, 4, 282, 1900000),
(15, 'Hotline Bling', 5, 5, 268, 1500000),
(16, 'Nordic Nights', 16, 16, 265, 22000),
(17, 'Pop Revolution', 17, 17, 220, 28000),
(18, 'Soundscapes', 18, 18, 255, 60000),
(19, 'Tanzanian Grooves', 19, 19, 245, 19000),
(20, 'Czech Melodies', 20, 20, 235, 7000),
(21, 'Indonesian Dream', 21, 21, 250, 65000),
(22, 'Mongolian Echoes', 22, 22, 240, 24000),
(23, 'Jazz Standards', 23, 23, 225, 50000),
(24, 'Japanese Melody', 24, 24, 210, 9000),
(25, 'Hip Hop Beats', 25, 25, 230, 17000),
(26, 'Venezuelan Sunshine', 26, 26, 260, 12000),
(27, 'Chad Rhythms', 27, 27, 195, 8000),
(28, 'Portuguese Jazz', 28, 28, 285, 35000),
(29, 'Indonesian Soul', 29, 29, 245, 45000),
(30, 'Timeless Hits', 30, 30, 235, 60000),
(31, 'China Nights', 31, 31, 255, 25000),
(32, 'Beats in the Night', 32, 32, 220, 18000),
(33, 'Heartfelt Harmonies', 33, 33, 240, 37000),
(34, 'Siburn Serenade', 34, 34, 230, 29000),
(35, 'Fusion Beats', 35, 35, 270, 41000);
```

```sql
INSERT INTO Playlists (PlaylistID, UserID, Name, CreationDate, PrivacyStatus) VALUES
(1, 1, 'My Favorite Rock Songs', '2023-04-01', 'Public'),
(2, 2, 'Pop Hits', '2022-08-15', 'Private'),
(3, 3, 'Chill Vibes', '2023-01-20', 'Unlisted'),
(4, 4, 'Workout Music', '2022-12-05', 'Public'),
(5, 5, 'Road Trip Anthems', '2023-05-10', 'Private'),
(6, 6, 'Hip Hop Classics', '2023-06-18', 'Public'),
(7, 7, 'Cuban Rhythms', '2023-07-22', 'Public'),
(8, 8, 'Late Night Grooves', '2023-08-15', 'Private'),
(9, 9, 'Peruvian Beats', '2023-09-02', 'Public'),
(10, 10, 'Acoustic Moments', '2023-10-10', 'Private'),
(11, 11, 'Rock Anthems', '2023-11-05', 'Public'),
(12, 12, 'Dance Party', '2023-12-01', 'Public'),
(13, 13, 'Indie Discoveries', '2023-01-20', 'Private'),
(14, 14, 'Mellow Moods', '2023-02-18', 'Public'),
(15, 15, 'World Sounds', '2023-03-27', 'Public'),
(16, 16, 'Throwback Jams', '2023-04-22', 'Private'),
(17, 17, 'Night Out', '2023-05-14', 'Public'),
(18, 18, 'Workout Mix', '2023-06-30', 'Private'),
(19, 19, 'Chill Out Lounge', '2023-07-15', 'Public'),
(20, 20, 'Weekend Vibes', '2023-08-10', 'Public'),
(21, 21, 'Jazz Classics', '2023-09-05', 'Private'),
(22, 22, 'Festival Favorites', '2023-10-25', 'Public'),
(23, 23, 'Feel Good Songs', '2023-11-15', 'Public'),
(24, 24, 'Soulful Sounds', '2023-12-12', 'Private'),
(25, 25, 'Late Night Chill', '2023-01-10', 'Public'),
(26, 26, 'Travel Tunes', '2023-02-20', 'Public'),
(27, 27, 'Epic Soundtracks', '2023-03-30', 'Private'),
(28, 28, 'Best of 2023', '2023-04-14', 'Public'),
(29, 29, 'Pop Faves', '2023-05-22', 'Public'),
(30, 30, 'Rock & Roll', '2023-06-16', 'Private');


INSERT INTO Subscriptions (SubscriptionID, UserID, PlanType, StartDate, EndDate,
AutoRenewal) VALUES
(1, 1, 'Premium', '2023-04-01', '2023-05-01', TRUE),
(2, 2, 'Basic', '2022-08-15', NULL, FALSE),
(3, 3, 'Standard', '2023-01-20', '2023-02-20', TRUE),
```

```
(4, 4, 'Premium', '2022-12-05', '2023-01-05', TRUE),
(5, 5, 'Basic', '2023-05-10', NULL, FALSE),
(6, 6, 'Basic', '2023-06-01', '2024-06-01', TRUE),
(7, 7, 'Premium', '2023-07-01', '2024-07-01', TRUE),
(8, 8, 'Standard', '2023-08-01', '2024-08-01', FALSE),
(9, 9, 'Basic', '2023-09-01', '2024-09-01', TRUE),
(10, 10, 'Premium', '2023-10-01', '2024-10-01', TRUE),
(11, 11, 'Standard', '2023-11-01', '2024-11-01', FALSE),
(12, 12, 'Basic', '2023-12-01', '2024-12-01', TRUE),
(13, 13, 'Premium', '2023-01-05', '2024-01-05', TRUE),
(14, 14, 'Standard', '2023-02-05', '2024-02-05', FALSE),
(15, 15, 'Basic', '2023-03-05', '2024-03-05', TRUE),
(16, 16, 'Premium', '2023-04-05', '2024-04-05', TRUE),
(17, 17, 'Standard', '2023-05-05', '2024-05-05', FALSE),
(18, 18, 'Basic', '2023-06-05', '2024-06-05', TRUE),
(19, 19, 'Premium', '2023-07-05', '2024-07-05', TRUE),
(20, 20, 'Standard', '2023-08-05', '2024-08-05', FALSE),
(21, 21, 'Basic', '2023-09-05', '2024-09-05', TRUE),
(22, 22, 'Premium', '2023-10-05', '2024-10-05', TRUE),
(23, 23, 'Standard', '2023-11-05', '2024-11-05', FALSE),
(24, 24, 'Basic', '2023-12-05', '2024-12-05', TRUE),
(25, 25, 'Premium', '2023-01-10', '2024-01-10', TRUE),
(26, 26, 'Standard', '2023-02-10', '2024-02-10', FALSE),
(27, 27, 'Basic', '2023-03-10', '2024-03-10', TRUE),
(28, 28, 'Premium', '2023-04-10', '2024-04-10', TRUE),
(29, 29, 'Standard', '2023-05-10', '2024-05-10', FALSE),
(30, 30, 'Basic', '2023-06-10', '2024-06-10', TRUE);

INSERT INTO Genres (GenreID, Name, Description) VALUES
(1, 'Rock', 'A broad genre of popular music that originated as "rock and roll" in
the United States in the late 1940s.'),
(2, 'Pop', 'A genre of popular music that originated in its modern form in the
United States and the United Kingdom.'),
(3, 'Hip Hop', 'A genre of popular music developed in the United States by inner-
city African Americans.'),
(4, 'K-pop', 'A genre of popular music originating in South Korea.'),
(5, 'Country', 'A genre of popular music that originated with blues, old-time music,
and various types of American folk.'),
(6, 'Jazz', 'A genre of music that originated from African American communities,
```

```
known for its swing and blue notes.'),
(7, 'Electronic', 'Music that employs electronic devices and technology,
encompassing various styles such as techno, house, and ambient.'),
(8, 'Reggae', 'A music genre that originated in Jamaica, known for its offbeat
rhythm and socially conscious lyrics.'),
(9, 'Classical', 'A broad term for a long tradition of music that includes many
different styles and periods, typically characterized by complex structures and
instrumentation.'),
(10, 'R&B', 'Rhythm and blues music that combines elements of soul, funk, pop, and
hip hop.'),
(11, 'Metal', 'A genre characterized by heavy guitar riffs, aggressive vocals, and a
strong emphasis on rhythm.'),
(12, 'Indie', 'A genre that typically refers to music produced independently from
major commercial record labels.'),
(13, 'Blues', 'A genre that originated from African American spirituals,
characterized by its melancholic sound and expressive lyrics.');


INSERT INTO Advertisements (AdID, Content, TargetAudience, Duration, SponsorName,
ImpressionsCount) VALUES
(1, 'New album release! Check it out now!', 'Music lovers', 30, 'Spotify', 10000),
(2, 'Get 50% off your first month!', 'New users', 15, 'Apple Music', 5000),
(3, 'Upgrade to Premium for ad-free music!', 'Free users', 20, 'Amazon Music',
7500),
(4, 'Discover the sounds of Helenelizabeth Borres!', 'Electronic Music Fans', 30,
'ElectroVibe', 12000),
(5, 'Catch Rossie Rapport's new single!', 'Pop Fans', 30, 'MusicNow', 18000),
(6, 'Experience the beats of Cherise Larway!', 'Jazz Lovers', 30, 'LiveMusic',
16000),
(7, 'Analiese St Clair's new album is out now!', 'Pop Fans', 30, 'ChartTopper',
22000),
(8, 'Greer Matuskiewicz - New Jazz Album!', 'Jazz Fans', 30, 'MusicSphere', 14000),
(9, 'Get ready for Mair Jowling's tour!', 'Jazz Audience', 30, 'JazzWorld', 15000),
(10, 'Ashley Pesik's electronic vibes await you!', 'Electronic Music Fans', 30,
'ElectroBuzz', 13000),
(11, 'Fianna McKerron's Rock album available now!', 'Rock Fans', 30, 'RockOn',
17000),
(12, 'Explore the rhythms of Arlette Guice!', 'Pop Music Lovers', 30, 'PopVibes',
19000),
```

```
(13, 'Loraine Grigoryov is taking over the charts!', 'Electronic Music Fans', 30,
'HitMaker', 21000),
(14, 'Sunny Brader's music will lift your spirits!', 'Rock Audience', 30,
'RockLife', 16000),
(15, 'Discover the magic of Larina Sacaze!', 'Hip Hop Fans', 30, 'HipHopCentral',
14000);

INSERT INTO PlaylistSongs (PlaylistID, SongID, AddedDate) VALUES
(1, 11, '2023-04-01'),
(2, 2, '2022-08-15'),
(2, 3, '2022-08-15'),
(3, 4, '2023-01-20'),
(1, 1, '2024-01-15'),
(1, 2, '2024-01-16'),
(1, 3, '2024-01-17'),
(2, 4, '2024-01-18'),
(2, 5, '2024-01-19'),
(2, 6, '2024-01-20'),
(3, 7, '2024-01-21'),
(3, 8, '2024-01-22'),
(3, 9, '2024-01-23'),
(4, 10, '2024-01-24'),
(4, 11, '2024-01-25'),
(5, 12, '2024-01-26'),
(5, 13, '2024-01-27'),
(6, 14, '2024-01-28'),
(6, 15, '2024-01-29'),
(7, 16, '2024-01-30'),
(7, 17, '2024-01-31'),
(8, 18, '2024-02-01'),
(8, 19, '2024-02-02'),
(9, 20, '2024-02-03'),
(9, 21, '2024-02-04');

INSERT INTO UserArtistFollows (UserID, ArtistID, FollowDate) VALUES
(1, 1, '2023-04-01'),
(1, 2, '2023-04-01'),
(2, 3, '2022-08-15'),
(3, 4, '2023-01-20'),
```

```sql
(4, 5, '2022-12-05'),
(1, 3, '2024-01-03'),
(2, 4, '2024-01-04'),
(2, 5, '2024-01-05'),
(3, 6, '2024-01-06'),
(3, 7, '2024-01-07'),
(4, 8, '2024-01-08'),
(4, 9, '2024-01-09'),
(5, 10, '2024-01-10'),
(5, 11, '2024-01-11'),
(6, 12, '2024-01-12'),
(6, 13, '2024-01-13'),
(7, 14, '2024-01-14'),
(7, 15, '2024-01-15'),
(8, 16, '2024-01-16'),
(8, 17, '2024-01-17'),
(9, 18, '2024-01-18'),
(9, 19, '2024-01-19'),
(10, 20, '2024-01-20'),
(10, 21, '2024-01-21');

INSERT INTO UserAdImpressions (AdID, UserID, ImpressionDate) VALUES
(1, 2, '2023-04-01'),
(1, 5, '2023-04-01'),
(2, 8, '2022-08-15'),
(3, 11, '2023-01-20'),
(3, 14, '2023-01-20'),
(1, 1, '2024-01-01'),
(2, 1, '2024-01-03'),
(2, 3, '2024-01-04'),
(3, 2, '2024-01-05'),
(3, 4, '2024-01-06'),
(4, 5, '2024-01-07'),
(4, 6, '2024-01-08'),
(5, 7, '2024-01-09'),
(5, 8, '2024-01-10'),
(6, 9, '2024-01-11'),
(6, 10, '2024-01-12'),
(7, 1, '2024-01-13'),
```

```sql
(7, 3, '2024-01-14'),
(8, 2, '2024-01-15'),
(8, 4, '2024-01-16'),
(9, 5, '2024-01-17'),
(9, 6, '2024-01-18'),
(10, 7, '2024-01-19'),
(10, 8, '2024-01-20');

INSERT INTO UserSongInteractions (UserID, SongID, InteractionType, InteractionDate,
Rating) VALUES
(1, 1, 'Play', '2023-04-01', NULL),
(1, 2, 'Like', '2023-04-05', NULL),
(2, 3, 'Rate', '2022-08-15', 4),
(3, 4, 'Play', '2023-01-20', NULL),
(4, 5, 'Like', '2022-12-10', NULL),
(1, 1, 'Like', '2024-01-01', 5),
(1, 2, 'Play', '2024-01-02', NULL),
(2, 3, 'Like', '2024-01-03', 4),
(2, 4, 'Dislike', '2024-01-04', 1),
(3, 5, 'Play', '2024-01-05', NULL),
(3, 6, 'Like', '2024-01-06', 5),
(4, 7, 'Play', '2024-01-07', NULL),
(4, 8, 'Like', '2024-01-08', 4),
(5, 9, 'Dislike', '2024-01-09', 2),
(5, 10, 'Like', '2024-01-10', 5),
(6, 11, 'Play', '2024-01-11', NULL),
(6, 12, 'Like', '2024-01-12', 3),
(7, 13, 'Play', '2024-01-13', NULL),
(7, 14, 'Dislike', '2024-01-14', 1),
(8, 15, 'Like', '2024-01-15', 5),
(8, 16, 'Play', '2024-01-16', NULL),
(9, 17, 'Like', '2024-01-17', 4),
(9, 18, 'Dislike', '2024-01-18', 2),
(10, 19, 'Play', '2024-01-19', NULL),
(10, 20, 'Like', '2024-01-20', 5);
```

Query 1: Aakash

- Write a query that retrieves the total number of songs each user has interacted with (either played, liked, or rated) and groups the results by the user. Include the user's name, the number of distinct songs, and order the results by the total interactions in descending order.

```sql
SELECT
    U.Username,
    COUNT(DISTINCT SongID) AS NumSongsInteracted
FROM
    Users U,
    UserSongInteractions USI
WHERE
    U.UserID = USI.UserID
    AND USI.InteractionType IN ('Play', 'Like', 'Rate')
GROUP BY
    U.UserID, U.Username
ORDER BY
    NumSongsInteracted DESC;
```

	username character varying (50)	numsongsinteracted bigint
1	popenthusiast789	3
2	jazzlover001	3
3	worldmusicguru678	2
4	hiphophead567	2
5	indielover012	2
6	musiclover123	2
7	classicalnut890	1
8	rockfan456	1
9	countryfan234	1
10	electronicfan345	1

Query 2: Arslan Bisharat

- Write a query that calculates the average rating for songs by each artist. Include the artist's name, the average rating, and order the results by the average rating in descending order.

```sql
SELECT
    A.ArtistName,
    ROUND(AVG(USI.Rating),2) AS AverageRating
FROM
    Artists A, Songs S, UserSongInteractions USI
WHERE
    A.ArtistID = S.ArtistID
    AND S.SongID = USI.SongID
    AND USI.Rating IS NOT NULL
GROUP BY
    A.ArtistID, A.ArtistName
ORDER BY
    AverageRating DESC;
```

Data Output Messages Notifications

	artistname character varying (100)	averagerating numeric
1	Billie Eilish	5.00
2	Adele	5.00
3	Esma Saben	5.00
4	Drake	5.00
5	The Beatles	5.00
6	BTS	4.00
7	Taylor Swift	4.00
8	Sylvan Jancey	4.00
9	Beyoncé	3.00
10	Veradis Ellen	2.00
11	Justin Bieber	2.00
12	Ed Sheeran	1.00

Query 3: Dominic Mattiello

- Write a query that retrieves the most popular genre based on the total number of song plays. Include the genre name and the total play count, and order the results by the play count in descending order.

```sql
SELECT
    G.Name AS GenreName,
    SUM(S.PlayCount) AS TotalPlayCount
FROM
    Genres G, Songs S, Albums A
WHERE
    S.AlbumID = A.AlbumID
    AND A.Genre = G.Name
GROUP BY
    G.GenreID, G.Name
ORDER BY
    TotalPlayCount DESC
```

Data Output Messages Notifications

	genrename character varying (50)	totalplaycount bigint
1	Pop	15956000
2	Hip Hop	3184000
3	Rock	2816000
4	K-pop	1700000
5	Jazz	130000
6	Electronic	125000

Query 4: Kashisq Patel

- Write a query that lists the song names played by the users who has a premium subscription plan and group them by the song title and songid. Order the result by the playcount in descending order.

```
SELECT
    s.title,
    COUNT(usi.InteractionType) AS PlayCount
FROM
    songs s, usersonginteractions usi, users u,
subscriptions sb

WHERE
    s.songid = usi.songid
      and usi.userid = u.userid
      and u.userid = sb.userid
      and sb.plantype = 'Premium'
GROUP BY
    s.title, s.songid
ORDER BY
    playcount DESC;
```

	title character varying (100)	playcount bigint
1	Formation	2
2	Here Comes the Sun	2
3	Work	1
4	Tanzanian Grooves	1
5	Czech Melodies	1
6	God's Plan	1
7	Thinking Out Loud	1
8	Fake Love	1
9	Blank Space	1

7. ORM Implementation

```
from sqlalchemy import create_engine, Column, Integer, String,
ForeignKey, Date
from sqlalchemy.orm import declarative_base, relationship, Session
from datetime import datetime
from typing import List

Base = declarative_base()
engine =
create_engine("postgresql://marslan:12345678@localhost:5432/project?o
ptions=-csearch_path%3Dpublic")

# Classes - Dominic (User, Playlist)
class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    name = Column(String(50), nullable=False)
    email = Column(String(100), nullable=False)
    # One-to-many: one user can have multiple playlists
    playlists: List["Playlist"] = relationship(
        'Playlist',
        back_populates='owner',
        collection_class=list
    )

    def __repr__(self):
        return f"User(id={self.id}, name='{self.name}',
email='{self.email}')"




class Playlist(Base):
    __tablename__ = 'playlists'
    id = Column(Integer, primary_key=True)
```

```
    name = Column(String(100), nullable=False)
    user_id = Column(Integer, ForeignKey('users.id'))
    # Many-to-one: many playlists belong to one user
    owner = relationship('User', back_populates='playlists')

    def __repr__(self):
        return f"Playlist(id={self.id}, name='{self.name}',
user_id={self.user_id})"

# Classes - Kanishq (Artist, Album)
class Artist(Base):
    __tablename__ = 'artists'
    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    genre = Column(String(30))
    country = Column(String(2))
    # One-to-many: one artist can have multiple albums
    albums: List["Album"] = relationship(
        'Album',
        back_populates='artist',
        collection_class=list
    )

    def __repr__(self):
        return f"Artist(id={self.id}, name='{self.name}')"

class Album(Base):
    __tablename__ = 'albums'
    id = Column(Integer, primary_key=True)
    title = Column(String(100), nullable=False)
    artist_id = Column(Integer, ForeignKey('artists.id'))
    release_date = Column(Date)
    genre = Column(String(30))
    # Many-to-one: many albums belong to one artist
    artist = relationship('Artist', back_populates='albums')
    # One-to-many: one album can have multiple songs
    songs: List["Song"] = relationship(
        'Song',
        back_populates='album',
```

```
        collection_class=list
    )

    def __repr__(self):
        return f"Album(id={self.id}, title='{self.title}')"

# Classes - Arslan (Song)
class Song(Base):
    __tablename__ = 'songs'
    id = Column(Integer, primary_key=True)
    title = Column(String(100), nullable=False)
    album_id = Column(Integer, ForeignKey('albums.id'))
    duration = Column(Integer)
    play_count = Column(Integer, default=0)
    # Many-to-one: many songs belong to one album
    album = relationship('Album', back_populates='songs')
    # One-to-many: one song can have multiple interactions
    interactions: List["UserSongInteraction"] = relationship(
        'UserSongInteraction',
        back_populates='song',
        collection_class=list
    )

    def __repr__(self):
        return f"Song(id={self.id}, title='{self.title}')"









# Classes - Aakash (UserSongInteraction)
class UserSongInteraction(Base):
   __tablename__ = 'user_song_interactions'
```

```
    user_id = Column(Integer, ForeignKey('users.id'),
primary_key=True)
    song_id = Column(Integer, ForeignKey('songs.id'),
primary_key=True)
    interaction_type = Column(String(10), nullable=False)
    interaction_date = Column(Date, default=datetime.now())
    rating = Column(Integer)
    # Many-to-one: many interactions belong to one user/song
    user: List["User"] = relationship(
        "User",
        back_populates="interactions",
        collection_class=list
    )
    song: List["Song"] = relationship(
        "Song",
        back_populates="interactions",
        collection_class=list
    )


    def __repr__(self):
        return f"UserSongInteraction(user_id={self.user_id},
song_id={self.song_id})"

def insert_sample_data(session):
    # Clear existing data
    session.query(UserSongInteraction).delete()
    session.query(Song).delete()
    session.query(Playlist).delete()
    session.query(Album).delete()
    session.query(Artist).delete()
    session.query(User).delete()
    session.commit()

    # Create 6 Users (to ensure at least 5 results)
    users = [
        User(name='John Doe', email='john@example.com'),
        User(name='Jane Smith', email='jane@example.com'),
        User(name='Mike Brown', email='mike@example.com'),
```

```
        User(name='Lisa White', email='lisa@example.com'),
        User(name='Tom Black', email='tom@example.com'),
        User(name='Anna Green', email='anna@example.com')
    ]
    session.add_all(users)

    # Create 6 Artists
    artists = [
        Artist(name='The Beatles', genre='Rock', country='UK'),
        Artist(name='Taylor Swift', genre='Pop', country='US'),
        Artist(name='Ed Sheeran', genre='Pop', country='UK'),
        Artist(name='BTS', genre='K-pop', country='KR'),
        Artist(name='Drake', genre='Hip-hop', country='CA'),
        Artist(name='Adele', genre='Pop', country='UK')
    ]
    session.add_all(artists)

    # Create multiple albums per artist (at least 6)
    albums = [
        Album(title='Abbey Road', artist=artists[0],
release_date=datetime(1969, 9, 26)),
        Album(title='1989', artist=artists[1],
release_date=datetime(2014, 10, 27)),
        Album(title='÷', artist=artists[2],
release_date=datetime(2017, 3, 3)),
        Album(title='Map of the Soul', artist=artists[3],
release_date=datetime(2020, 2, 21)),
        Album(title='Scorpion', artist=artists[4],
release_date=datetime(2018, 6, 29)),
        Album(title='25', artist=artists[5],
release_date=datetime(2015, 11, 20))
    ]
    session.add_all(albums)

    # Create multiple songs per album (at least 6)
    songs = [
        Song(title='Here Comes the Sun', album=albums[0],
duration=185, play_count=1000000),
        Song(title='Shake It Off', album=albums[1], duration=219,
```

```
play_count=2000000),
        Song(title='Shape of You', album=albums[2], duration=233,
play_count=3000000),
        Song(title='Dynamite', album=albums[3], duration=199,
play_count=1500000),
        Song(title='God\'s Plan', album=albums[4], duration=198,
play_count=2500000),
        Song(title='Hello', album=albums[5], duration=295,
play_count=1800000)
    ]
    session.add_all(songs)

    # Create multiple playlists per user (at least 6)
    playlists = [
        Playlist(name='Rock Mix', owner=users[0]),
        Playlist(name='Pop Hits', owner=users[1]),
        Playlist(name='Study Music', owner=users[2]),
        Playlist(name='Workout Mix', owner=users[3]),
        Playlist(name='Chill Vibes', owner=users[4]),
        Playlist(name='Party Mix', owner=users[5])
    ]
    session.add_all(playlists)

    # Create multiple interactions (at least 6)
    interactions = [
        UserSongInteraction(user=users[0], song=songs[0],
interaction_type='Like', rating=5),
        UserSongInteraction(user=users[1], song=songs[1],
interaction_type='Play', rating=4),
        UserSongInteraction(user=users[2], song=songs[2],
interaction_type='Rate', rating=3),
        UserSongInteraction(user=users[3], song=songs[3],
interaction_type='Like', rating=5),
        UserSongInteraction(user=users[4], song=songs[4],
interaction_type='Play', rating=4),
        UserSongInteraction(user=users[5], song=songs[5],
interaction_type='Rate', rating=5)
    ]
    session.add_all(interactions)
```

```
    session.commit()

# Original queries modified to ensure at least 5 results
def query_dominic(session):
    print("\nDominic's Query - Number of songs each user has
interacted with:")
    print("-----------------------------------------------------------
--")
    results = session.query(User.name,
                            session.query(UserSongInteraction)
                            .filter(UserSongInteraction.user_id ==
User.id)

.filter(UserSongInteraction.interaction_type.in_(['Play', 'Like',
'Rate']))

.with_entities(session.func.count(UserSongInteraction.song_id.distinc
t()))
                            .scalar_subquery()
                            .label('num_songs_interacted'))\
                        .order_by(session.desc('num_songs_interacted'))\
                        .limit(6)\
                        .all()

    for username, num_interactions in results:
        print(f"User: {username}")
        print(f"Number of songs interacted with: {num_interactions}")
        print("---")

def query_kanishq(session):
    print("\nKanishq's Query - Average rating for songs by each
artist:")
    print("-----------------------------------------------------")
    results = session.query(
        Artist.name,

session.func.round(session.func.avg(UserSongInteraction.rating),
2).label('avg_rating')
    ).join(Album).join(Song).join(UserSongInteraction)\
```

```
     .group_by(Artist.name)\
     .order_by(session.desc('avg_rating'))\
     .limit(6)\
     .all()

    for artist_name, avg_rating in results:
        print(f"Artist: {artist_name}")
        print(f"Average Rating: {avg_rating}")
        print("---")

def query_arslan(session):
    print("\nArslan's Query - Most popular genres based on play
count:")
    print("-----------------------------------------------------")
    results = session.query(
        Album.genre,
        session.func.sum(Song.play_count).label('total_plays')
    ).join(Song)\
     .group_by(Album.genre)\
     .order_by(session.desc('total_plays'))\
     .limit(6)\
     .all()

    for genre, total_plays in results:
        print(f"Genre: {genre}")
        print(f"Total Plays: {total_plays}")
        print("---")

def query_aakash(session):
    print("\nAakash's Query - Songs played by premium users:")
    print("--------------------------------------------")
    results = session.query(
        Song.title,

session.func.count(UserSongInteraction.user_id).label('play_count')
    ).join(UserSongInteraction)\
     .join(User)\
     .group_by(Song.id, Song.title)\
     .order_by(session.desc('play_count'))\
```

```
     .limit(6)\
     .all()

    for song_title, play_count in results:
        print(f"Song: {song_title}")
        print(f"Number of Plays: {play_count}")
        print("---")

if __name__ == "__main__":
    print("Executing ORM Queries")
    print("=====================")
    session = Session(engine)

    try:
        # Create tables
        Base.metadata.drop_all(engine)
        Base.metadata.create_all(engine)

        # Insert sample data
        insert_sample_data(session)

        while True:
            print("\nChoose a query to execute:")
            print("1. Dominic's Query - Users and song interactions")
            print("2. Kanishq's Query - Artist ratings")
            print("3. Arslan's Query - Genre popularity")
            print("4. Aakash's Query - Premium user song plays")
            print("5. Exit")
            choice = input("Enter your choice (1-5): ")

            if choice == "1":
                query_dominic(session)
            elif choice == "2":
                query_kanishq(session)
            elif choice == "3":
                query_arslan(session)
            elif choice == "4":
                query_aakash(session)
            elif choice == "5":
```

```
                print("Exiting program. Goodbye!")
                break
            else:
                print("Invalid choice. Please select 1-5.")
    except Exception as e:
        print(f"Error: {e}")
        session.rollback()
    finally:
        session.close()
```

Query 1 - Dominic : Retrieves users and their associated playlists in a one-to-many relationship, showing each user's name and the playlists they have created, sorted alphabetically.

```
def query_dominic(session):
    print("\nDominic's Query - Users and their playlists:")
    print("--------------------------------------------")
    results = session.query(User, Playlist) \
        .join(Playlist) \
        .order_by(User.name) \
        .limit(5).all()
    for user, playlist in results:
        print(f"User: {user.name}")
        print(f"  - Playlist: {playlist.name}")
```

```
Choose a query to execute:
1. Dominic's Query - Users and Playlists
2. Kanishq's Query - Artists and Albums
3. Arslan's Query - Albums and Songs
4. Aakash's Query - User Interactions
5. Exit
Enter your choice (1-5): 1

Dominic's Query - Users and their playlists:
--------------------------------------------
User: Jane Smith
  - Playlist: Pop Hits
User: John Doe
  - Playlist: Rock Mix
User: Lisa White
  - Playlist: Workout Mix
User: Mike Brown
  - Playlist: Study Music
User: Tom Black
  - Playlist: Chill Vibes
```

Query 2: Kanishq: Lists artists and the albums they have released, representing a one-to-many relationship, detailing each artist's name along with their album titles, sorted by artist name.

```python
def query_kanishq(session):
    print("\nKanishq's Query - Artists and their albums:")
    print("-----------------------------------------")
    results = session.query(Artist, Album) \
        .join(Album) \
        .order_by(Artist.name) \
        .limit(5).all()
    for artist, album in results:
        print(f"Artist: {artist.name}")
        print(f"  - Album: {album.title}")
```

```
Choose a query to execute:
1. Dominic's Query - Users and Playlists
2. Kanishq's Query - Artists and Albums
3. Arslan's Query - Albums and Songs
4. Aakash's Query - User Interactions
5. Exit
Enter your choice (1-5): 2

Kanishq's Query - Artists and their albums:
-----------------------------------------
Artist: BTS
  - Album: Map of the Soul
Artist: Drake
  - Album: Scorpion
Artist: Ed Sheeran
  - Album: ÷
Artist: Taylor Swift
  - Album: 1989
Artist: The Beatles
  - Album: Abbey Road
```

Query 3: Arslan: Fetches albums and the songs they contain, highlighting the one-to-many relationship between albums and their tracks, including song duration and play count, ordered by album title.

```python
def query_arslan(session):
    print("\nArslan's Query - Albums and their songs:")
    print("----------------------------------------")
    results = session.query(Album, Song) \
        .join(Song) \
        .order_by(Album.title) \
        .limit(5).all()
    for album, song in results:
        print(f"Album: {album.title}")
        print(f"  - Song: {song.title}")
        print(f"    Duration: {song.duration} seconds")
        print(f"    Plays: {song.play_count}")
```

```
Choose a query to execute:
1. Dominic's Query - Users and Playlists
2. Kanishq's Query - Artists and Albums
3. Arslan's Query - Albums and Songs
4. Aakash's Query - User Interactions
5. Exit
Enter your choice (1-5): 3

Arslan's Query - Albums and their songs:
----------------------------------------
Album: ÷
  - Song: Shape of You
    Duration: 233 seconds
    Plays: 3000000
Album: 1989
  - Song: Shake It Off
    Duration: 219 seconds
    Plays: 2000000
Album: Abbey Road
  - Song: Here Comes the Sun
    Duration: 185 seconds
    Plays: 1000000
Album: Map of the Soul
  - Song: Dynamite
    Duration: 199 seconds
    Plays: 1500000
Album: Scorpion
  - Song: God's Plan
    Duration: 198 seconds
    Plays: 2500000
```

Query 4: Aakash: Fetches albums and the songs they contain, highlighting the one-to-many

relationship between albums and their tracks, including song duration and play count, ordered by album title.

```python
def query_aakash(session):
    print("\nAakash's Query - User interactions with songs:")
    print("----------------------------------------------")
    results = session.query(User, Song, UserSongInteraction) \
        .join(UserSongInteraction, User.id == UserSongInteraction.user_id) \
        .join(Song, Song.id == UserSongInteraction.song_id) \
        .order_by(User.name) \
        .limit(5).all()
    for user, song, interaction in results:
        print(f"User: {user.name}")
        print(f"  - Song: {song.title}")
        print(f"  - Interaction: {interaction.interaction_type}")
        if interaction.rating:
            print(f"  - Rating: {interaction.rating}")
```

```
Choose a query to execute:
1. Dominic's Query - Users and Playlists
2. Kanishq's Query - Artists and Albums
3. Arslan's Query - Albums and Songs
4. Aakash's Query - User Interactions
5. Exit
Enter your choice (1-5): 4

Aakash's Query - User interactions with songs:
----------------------------------------------
User: Jane Smith
  - Song: Shake It Off
  - Interaction: Play
User: John Doe
  - Song: Here Comes the Sun
  - Interaction: Like
  - Rating: 5
User: Lisa White
  - Song: Dynamite
  - Interaction: Like
  - Rating: 5
User: Mike Brown
  - Song: Shape of You
  - Interaction: Rate
  - Rating: 4
User: Tom Black
  - Song: God's Plan
  - Interaction: Play
```

GROUP STATUS REPORT

GROUP #: 1 GROUP NAME: DKAA Crew PHASE #: 3

Dates & attendance at group meetings in this phase:

Friday, Nov 1, 2024	03:45- 4:15	All group members present
Sunday, Nov 10, 2024	03:45- 4:15	All group members present
Tuesday, Nov 19, 2024	03:45- 4:15	All group members present
Wednesday, Nov 20, 2024	09:00- 10:00	All group members present

Overview of progress on the project as of October 24:

Phase 3 Deliverable is complete. The ORM is complete and functional. Listed below are the reports, queries, and responsibilities completed by each group member.

CONTRIBUTIONS OF GROUP MEMBERS

Dominic Mattiello (Leader):

- Designed and implemented query 1; retrieving users and their playlists.
- Ensured the one-to-many relationship between users and playlists classes is correct.
- Managed overall project progress and team coordination.

Kanishq Patel (Technical Advisor):

- Developed the query 2; showcasing artists and their albums.

- Highlighted the one-to-many relationship between artists and albums in results.
- Provided technical guidance and support to the team.

Arslan Bisharat (Phase Checker):

- Created the query 3; linking albums with their songs, emphasizing the one-to-many relationship.
- Validated the results for consistency and accuracy.
- Reviewed and ensured all deliverables met project requirements.

Aakash Dawani (Recorder):

- Implemented the query 4; displaying user interactions with songs.
- Focused on user-song relationships with attributes like interaction type and rating.
- Documented all phases and team contributions for submission.